Name : Anuj Thakre

Section : A_A2

Roll no. : 20

ML LAB

In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
df=pd.read_csv("USA_Housing.csv")
df
```

Out[2]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Addres
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	208 Michael Ferry Ap 674\nLaurabury, N 3701.
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	188 Johnson View Suite 079\nLak Kathleen, CA.
2	61287.067179	5.865890	8.512727	5.13	36882.159400	1.058988e+06	9127 Elizabet Stravenue\nDanieltowr WI 06482.
3	63345.240046	7.188236	5.586729	3.26	34310.242831	1.260617e+06	USS Barnett\nFPO A 4482
4	59982.197226	5.040555	7.839388	4.23	26354.109472	6.309435e+05	USNS Raymond\nFP(AE 0938
...	...	...	...	...	...	...	.
4995	60567.944140	7.830362	6.137356	3.46	22837.361035	1.060194e+06	USNS Williams\nFP(AP 30153-765
4996	78491.275435	6.999135	6.576763	4.02	25616.115489	1.482618e+06	PSC 9258, Bo 8489\nAPO AA 42991 335
4997	63390.686886	7.250591	4.805081	2.13	33266.145490	1.030730e+06	4215 Tracy Garde Suite 076\nJoshualanc VA 01.
4998	68001.331235	5.534388	7.130144	5.44	42625.620156	1.198657e+06	USS Wallace\nFPO A 7331

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Addres
4999	65510.581804	5.992305	6.792336	4.07	46501.283803	1.298950e+06	37778 George Ridge Apt. 509\nEast Holly NV 2.

5000 rows × 7 columns

In [3]:
```
df.head()
```

Out[3]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Address
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	188 Johnson Views Suite 079\nLake Kathleen, CA...
2	61287.067179	5.865890	8.512727	5.13	36882.159400	1.058988e+06	9127 Elizabeth Stravenue\nDanieltown, WI 06482...
3	63345.240046	7.188236	5.586729	3.26	34310.242831	1.260617e+06	USS Barnett\nFPO AP 44820
4	59982.197226	5.040555	7.839388	4.23	26354.109472	6.309435e+05	USNS Raymond\nFPO AE 09386

In [4]:
```
df.tail()
```

Out[4]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Address
4995	60567.944140	7.830362	6.137356	3.46	22837.361035	1.060194e+06	USNS Williams\nFPO AP 30153-7653
4996	78491.275435	6.999135	6.576763	4.02	25616.115489	1.482618e+06	PSC 9258, Box 8489\nAPO AA 42991-3352
4997	63390.686886	7.250591	4.805081	2.13	33266.145490	1.030730e+06	4215 Tracy Garden Suite 076\nJoshualand, VA 01...
4998	68001.331235	5.534388	7.130144	5.44	42625.620156	1.198657e+06	USS Wallace\nFPO AE

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Address
							73316
4999	65510.581804	5.992305	6.792336	4.07	46501.283803	1.298950e+06	37778 George Ridges Apt. 509\nEast Holly, NV 2...

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column                        Non-Null Count  Dtype  
---  ------                        --------------  -----  
 0   Avg. Area Income              5000 non-null   float64
 1   Avg. Area House Age           5000 non-null   float64
 2   Avg. Area Number of Rooms     5000 non-null   float64
 3   Avg. Area Number of Bedrooms  5000 non-null   float64
 4   Area Population               5000 non-null   float64
 5   Price                         5000 non-null   float64
 6   Address                       5000 non-null   object 
dtypes: float64(6), object(1)
memory usage: 273.6+ KB
```

In [6]:
```
df.describe()
```

Out[6]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5.000000e+03
mean	68583.108984	5.977222	6.987792	3.981330	36163.516039	1.232073e+06
std	10657.991214	0.991456	1.005833	1.234137	9925.650114	3.531176e+05
min	17796.631190	2.644304	3.236194	2.000000	172.610686	1.593866e+04
25%	61480.562388	5.322283	6.299250	3.140000	29403.928702	9.975771e+05
50%	68804.286404	5.970429	7.002902	4.050000	36199.406689	1.232669e+06
75%	75783.338666	6.650808	7.665871	4.490000	42861.290769	1.471210e+06
max	107701.748378	9.519088	10.759588	6.500000	69621.713378	2.469066e+06

In [7]:
```
pd.set_option('display.float_format', '{:.2f}'.format)
```

In [8]:
```
df.drop("Address",axis=1,inplace=True)
```

In [9]:
```
df
```

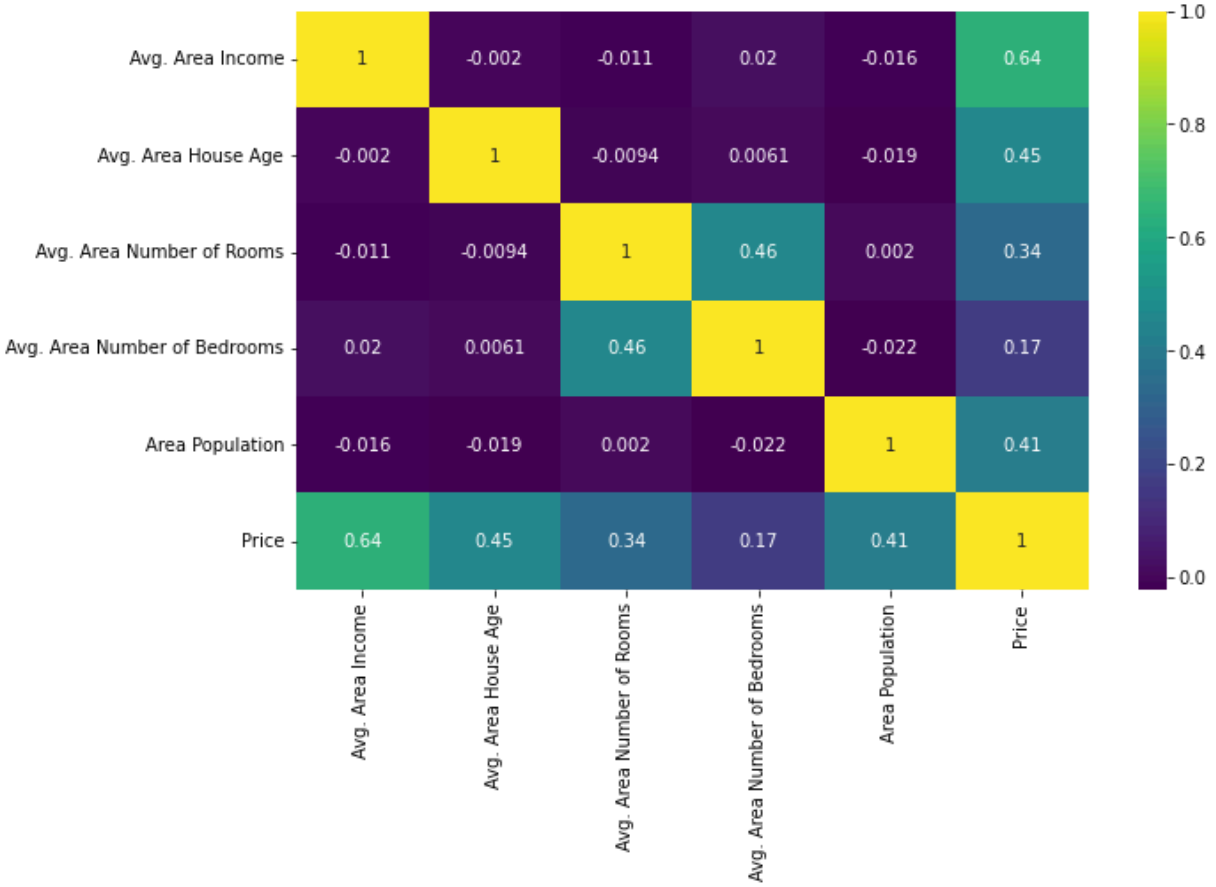
Out[9]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
0	79545.46	5.68	7.01	4.09	23086.80	1059033.56
1	79248.64	6.00	6.73	3.09	40173.07	1505890.91
2	61287.07	5.87	8.51	5.13	36882.16	1058987.99
3	63345.24	7.19	5.59	3.26	34310.24	1260616.81
4	59982.20	5.04	7.84	4.23	26354.11	630943.49
...	...	...	...	...	...	...
4995	60567.94	7.83	6.14	3.46	22837.36	1060193.79
4996	78491.28	7.00	6.58	4.02	25616.12	1482617.73
4997	63390.69	7.25	4.81	2.13	33266.15	1030729.58
4998	68001.33	5.53	7.13	5.44	42625.62	1198656.87
4999	65510.58	5.99	6.79	4.07	46501.28	1298950.48

5000 rows × 6 columns

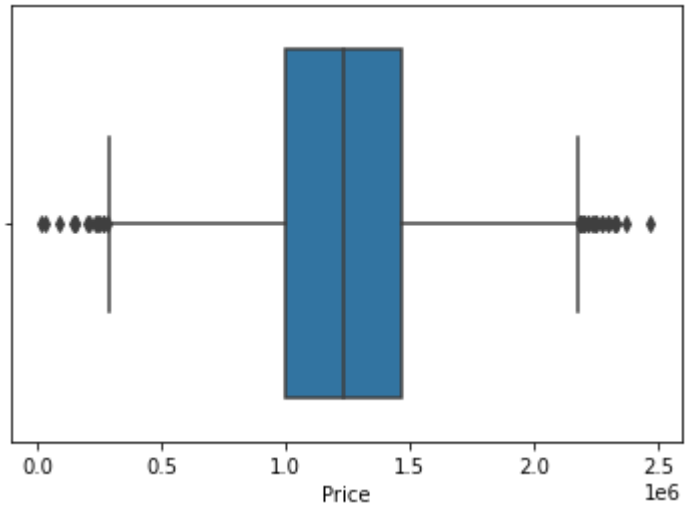
In [10]:

```
plt.figure(figsize=(10,6))
numeric_df=df.select_dtypes(include=['number'])
sns.heatmap(numeric_df.corr(),annot=True,cmap="viridis")
plt.show()
```



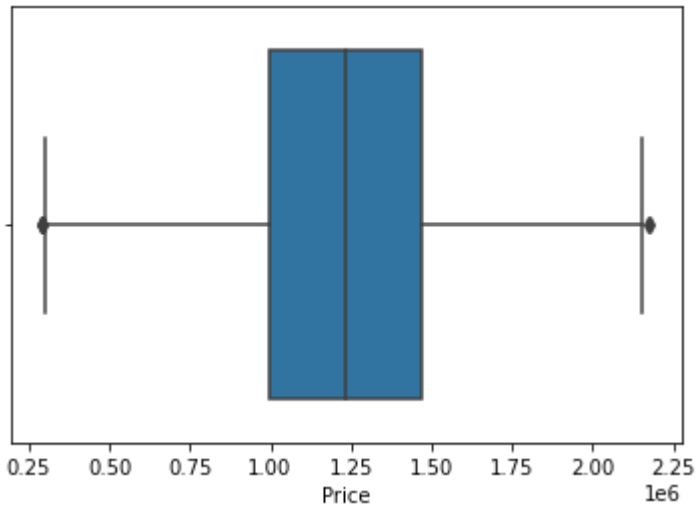
In [11]:

```
sns.boxplot(x=df["Price"])
plt.show()
```



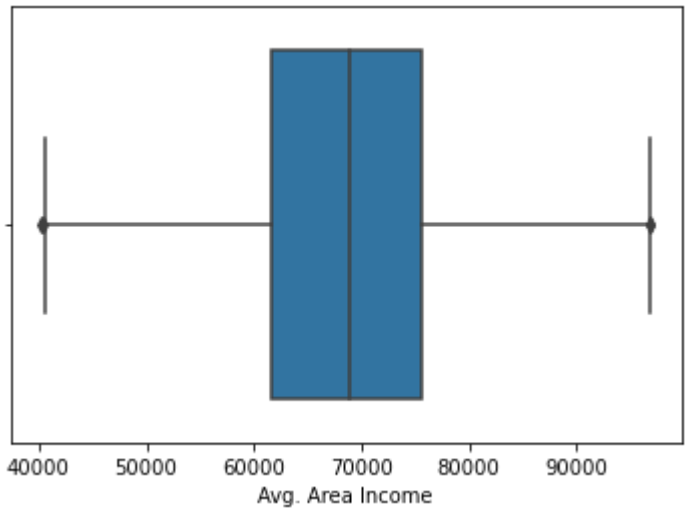
In [12]:
```
Q1=df["Price"].quantile(0.25)
Q3=df["Price"].quantile(0.75)

IQR=Q3-Q1

lower=Q1-1.5*IQR
upper=Q3+1.5*IQR

df=df[(df["Price"]>=lower)&(df["Price"]<=upper)]
```

In [13]:
```
sns.boxplot(x=df["Price"])
plt.show()
```

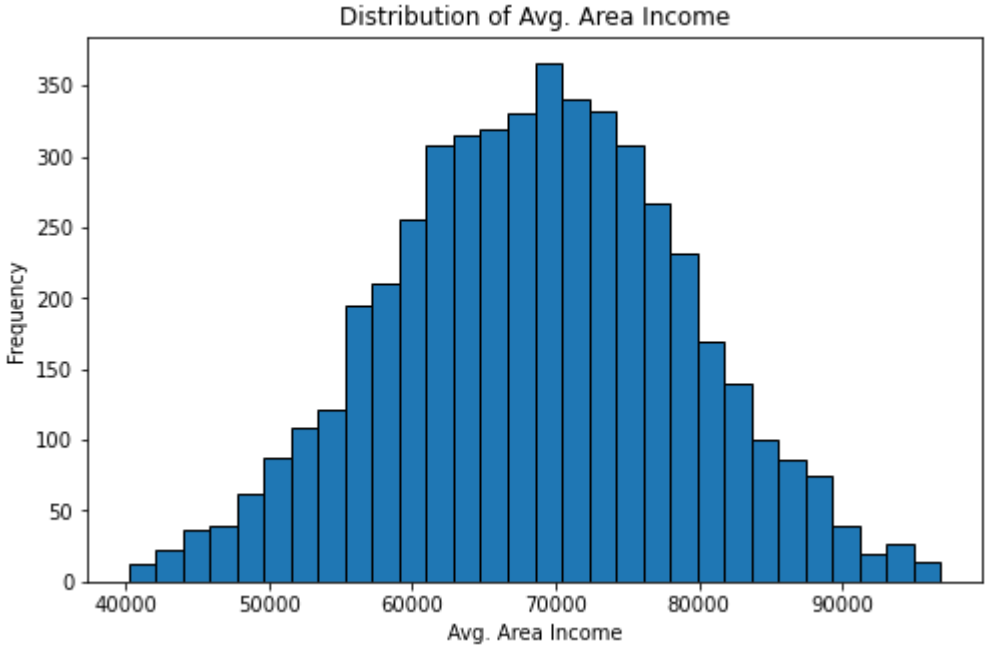


In [14]:
```
Q1=df["Avg. Area Income"].quantile(0.25)
Q3=df["Avg. Area Income"].quantile(0.75)

IQR=Q3-Q1

lower=Q1-1.5*IQR
upper=Q3+1.5*IQR

df=df[(df["Avg. Area Income"]>=lower) & (df["Avg. Area Income"]<=upper)]
```

In [15]:
```
sns.boxplot(x=df["Avg. Area Income"])
plt.show()
```



In [16]:
```
plt.figure(figsize=(8,5))
plt.hist(df['Avg. Area Income'], bins=30, edgecolor='black')
plt.title('Distribution of Avg. Area Income')
plt.xlabel('Avg. Area Income')
plt.ylabel('Frequency')
plt.show()
```



Simple Linear Regression

In [17]:
```
X=df[["Avg. Area Income"]]
y=df["Price"]
```

In [18]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test =train_test_split(X,y,test_size=0.2,random_state=42)
```

In [19]:
```
from sklearn.linear_model import LinearRegression
slr_model=LinearRegression()
slr_model.fit(X_train,y_train)
```

Out[19]:
```
LinearRegression()
```

In [20]:
```
y_pred=slr_model.predict(X_test)
```

In [21]:
```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np

print("\nSimple Linear Regression Result")
print("..............................")
print("MAE  :", mean_absolute_error(y_test, y_pred))
print("MSE  :", mean_squared_error(y_test, y_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score :", r2_score(y_test, y_pred))
```

```
Simple Linear Regression Result
..............................
MAE  : 209409.3018216627
MSE  : 66978557287.983284
RMSE : 258802.1585844741
R2 Score : 0.42914402441550337
```

Multi linear regression

In [22]:
```
X = df.drop("Price", axis=1)
y = df["Price"]
```

In [23]:
```
X = df.drop(["Price", "Address"], axis=1, errors="ignore")
y = df["Price"]
```

In [24]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [25]:
```
from sklearn.linear_model import LinearRegression

mlr_model = LinearRegression()
mlr_model.fit(X_train, y_train)

mlr_model
```

Out[25]:
```
LinearRegression()
```

In [26]:
```
y_pred = mlr_model.predict(X_test)
```

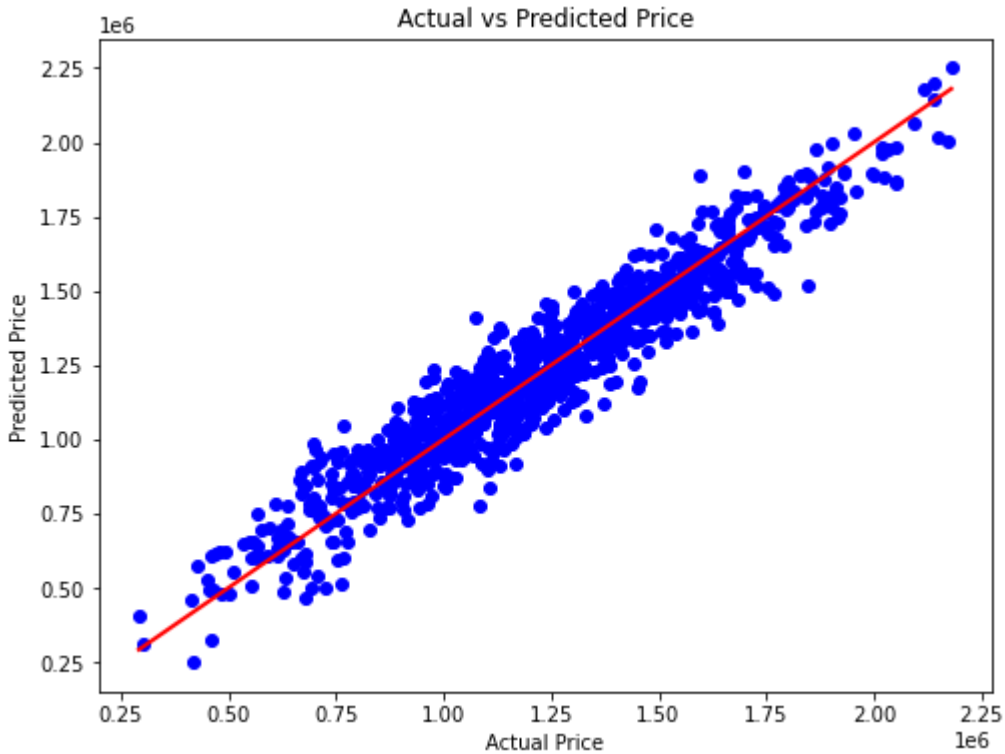
In [27]:
```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
```

```
from sklearn.metrics import r2_score
import numpy as np

print("\nMultiple Linear Regression Result")
print("..............................")
print("MAE :", mean_absolute_error(y_test, y_pred))
print("MSE :", mean_squared_error(y_test, y_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score :", r2_score(y_test, y_pred))
```

```
Multiple Linear Regression Result
..............................
MAE : 79279.44669409566
MSE : 9993387098.255205
RMSE : 99966.93002315919
R2 Score : 0.9148267001804845
```

In [28]:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(y_test, y_pred, color="blue")

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Price")

# Perfect prediction line
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red",
    linewidth=2
)

plt.show()
```



In [30]:
```
import numpy as np

# Take input from the user
income = float(input("Enter Average Area Income: "))
house_age = float(input("Enter Average Area House Age: "))
rooms = float(input("Enter Average Area Number of Rooms: "))
bedrooms = float(input("Enter Average Area Number of Bedrooms: "))
population = float(input("Enter Area Population: "))

# Create Input Array
test_input = np.array([[income, house_age, rooms, bedrooms, population]])

# Predict Price
prediction_price = mlr_model.predict(test_input)

# Display result
print("\nPredicted House Price : ${:,.2f}".format(prediction_price[0]))
```

```
Enter Average Area Income: 55
Enter Average Area House Age: 12
Enter Average Area Number of Rooms: 42
Enter Average Area Number of Bedrooms: 22
Enter Area Population: 12345

Predicted House Price : $4,641,652.37
```

Ridge regression

In [31]:
```
from sklearn.model_selection import GridSearchCV
```

In [32]:
```
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score

ridge = Ridge()

ridge.fit(X_train, y_train)

ridge_pred = ridge.predict(X_test)

print("Default Ridge R2 :", r2_score(y_test, ridge_pred))
```

```
Default Ridge R2 : 0.914832076303878
```

In [35]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1, 10]
}

grid_ridge = GridSearchCV(
    estimator=Ridge(),
    param_grid=param_grid,
    scoring='r2',
    cv=5
)

grid_ridge.fit(X_train, y_train)
```

```
print("Best alpha:", grid_ridge.best_params_)
print("Best R2 Score:", grid_ridge.best_score_)
```

```
Best alpha: {'alpha': 1}
Best R2 Score: 0.9121212051696579
```

In [36]:

```
best_ridge = grid_ridge.best_estimator_

ridge_pred = best_ridge.predict(X_test)
```

In [33]:

```
print("MAE :", mean_absolute_error(y_test, ridge_pred))
print("MSE :", mean_squared_error(y_test, ridge_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test, ridge_pred)))
print("R2 Score :", r2_score(y_test, ridge_pred))
```

```
MAE : 79277.25338222191
MSE : 9992756317.455666
RMSE : 99963.77502603464
R2 Score : 0.914832076303878
```

Lasso Regression

In [37]:

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score

param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1, 10]
}

grid_lasso = GridSearchCV(
    estimator=Lasso(max_iter=5000),
    param_grid=param_grid,
    cv=5,
    scoring='r2'
)

# Train on training data
grid_lasso.fit(X_train, y_train)

print("Best Parameters:", grid_lasso.best_params_)

# Predict on test data
lasso_pred = grid_lasso.predict(X_test)

print("R2 Score:", r2_score(y_test, lasso_pred))
```

```
Best Parameters: {'alpha': 10}
R2 Score: 0.9148280193164746
```

Model Predictions

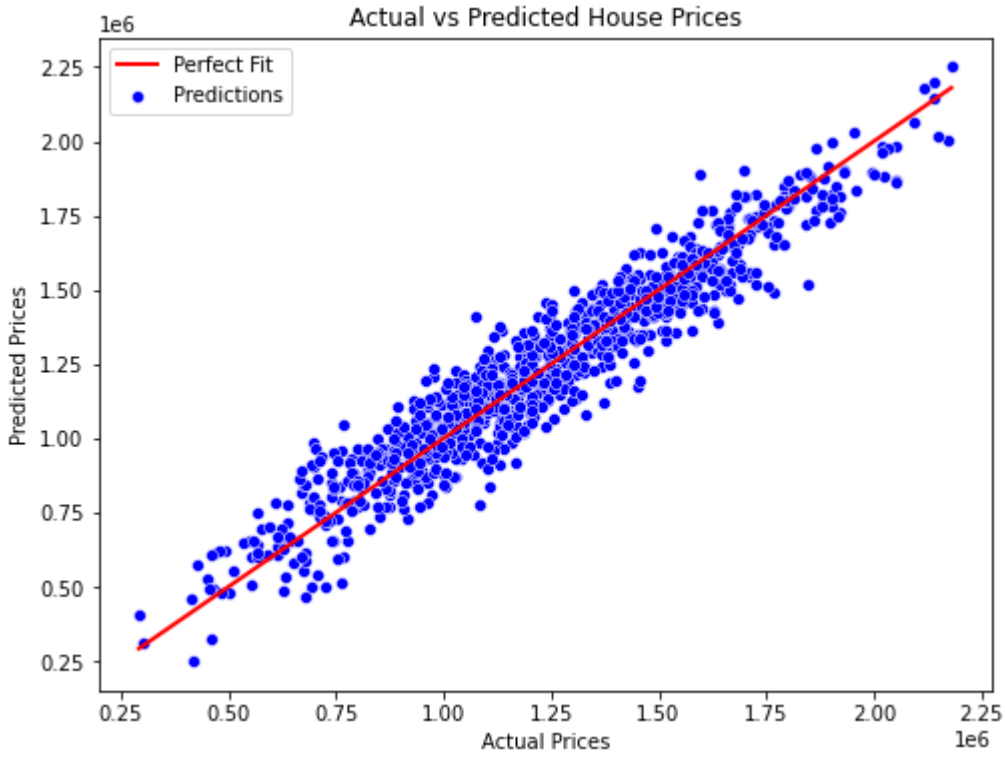
In [38]:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)
```

Out[38]:
```
LinearRegression()
```

In [39]:
```
predictions = model.predict(X_test)
```

In [40]:
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,6))

sns.scatterplot(
    x=y_test,
    y=predictions,
    color='blue',
    label='Predictions'
)


plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color='red',
    linewidth=2,
    label='Perfect Fit'
)

plt.title("Actual vs Predicted House Prices")
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")

plt.legend()

plt.show()
```



In [ ]:
```

```

In [ ]: